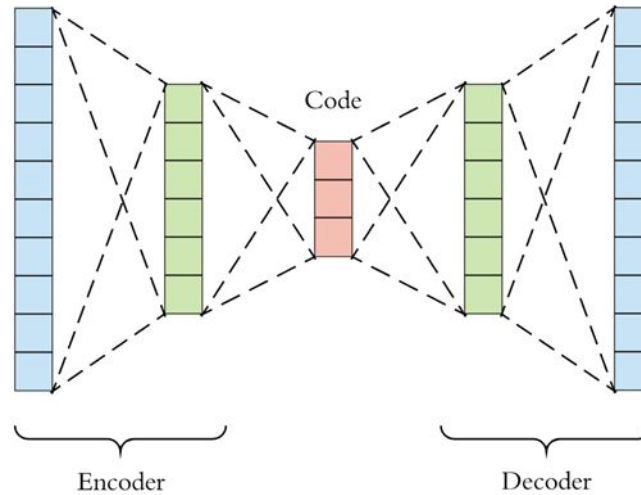


Artificial Intelligence and Machine Learning. 6CS012



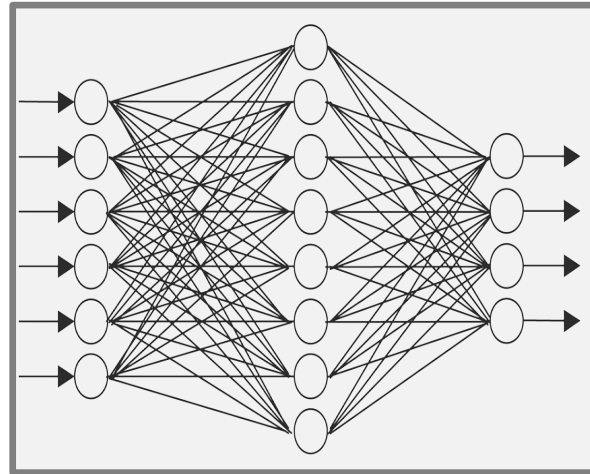
Lecture-07:AutoEncoders

Representational Learning: **Supervised Learning**

We train the two networks by minimizing the loss function (**cross entropy loss**)

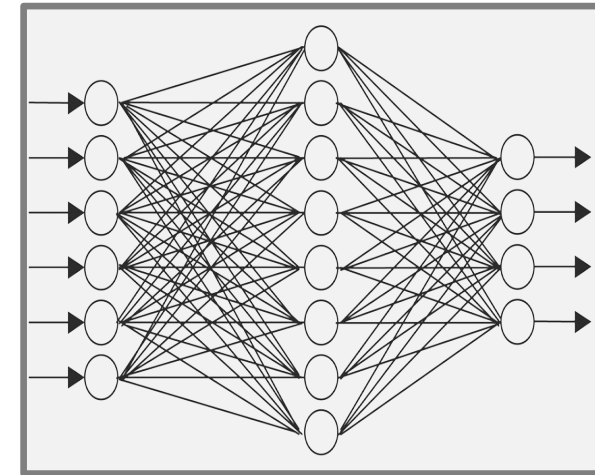
$$\mathcal{L}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, \hat{y}_i)$$

X



Feature Discovery Network

Features
 $\phi(x)$



Classification Network



Y
 $\{\text{Cat}, \text{Dog}\}$

Representational Learning: **Un/self-supervised Learning**

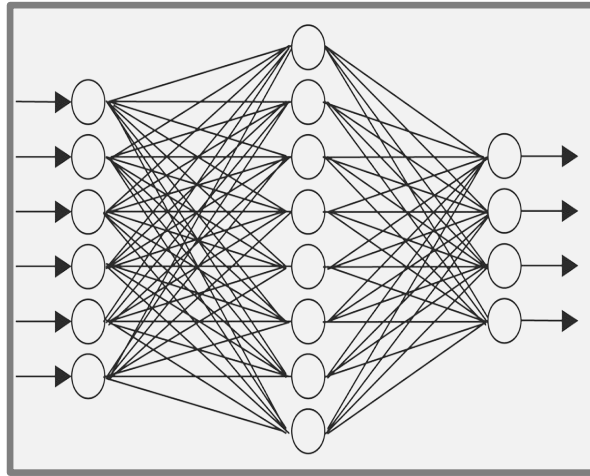
We train the two networks by minimizing the **reconstruction** loss function:

$$\mathcal{L} = \sum (x_i - \hat{x}_i)^2$$

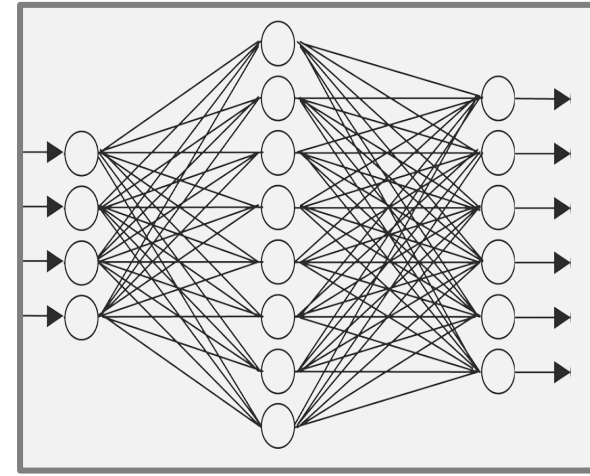
X

Auto encoder

$\hat{X}$



Features
 $\phi(x)$



Feature Discovery Network

Latent Space

Second Network

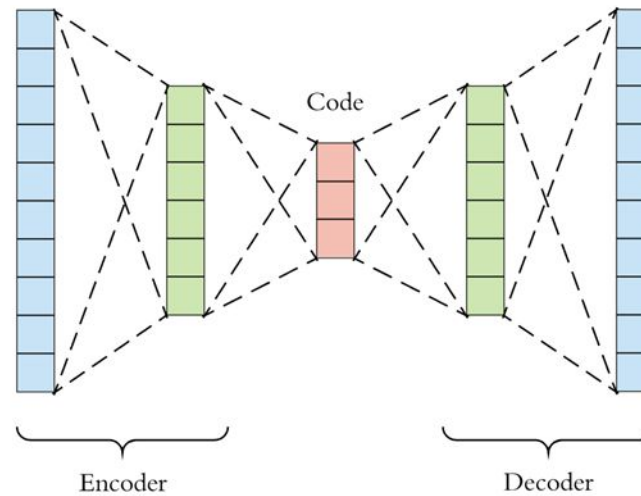
ENCODER

Z

DECODER

This is an **autoencoder**. It gets that name because it automatically finds the best way to encode the input so that the decoded version is as close as possible to the input.

Encoder-Decoder Architecture.



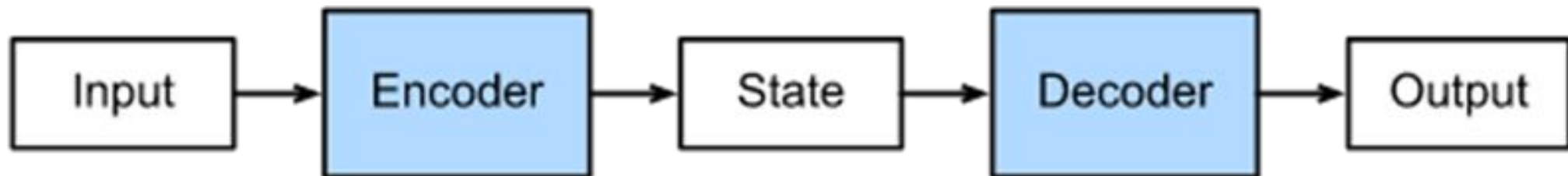
Introduction to Encoder – Decoder Architecture.

Encoder

- This process allows the encoder to **capture the most relevant information {feature}** from the inputs and produce a **fixed-size representation** of it.
- Encodes the important feature information in some latent dimension {generally smaller than input}
- Input can be image or text or others.

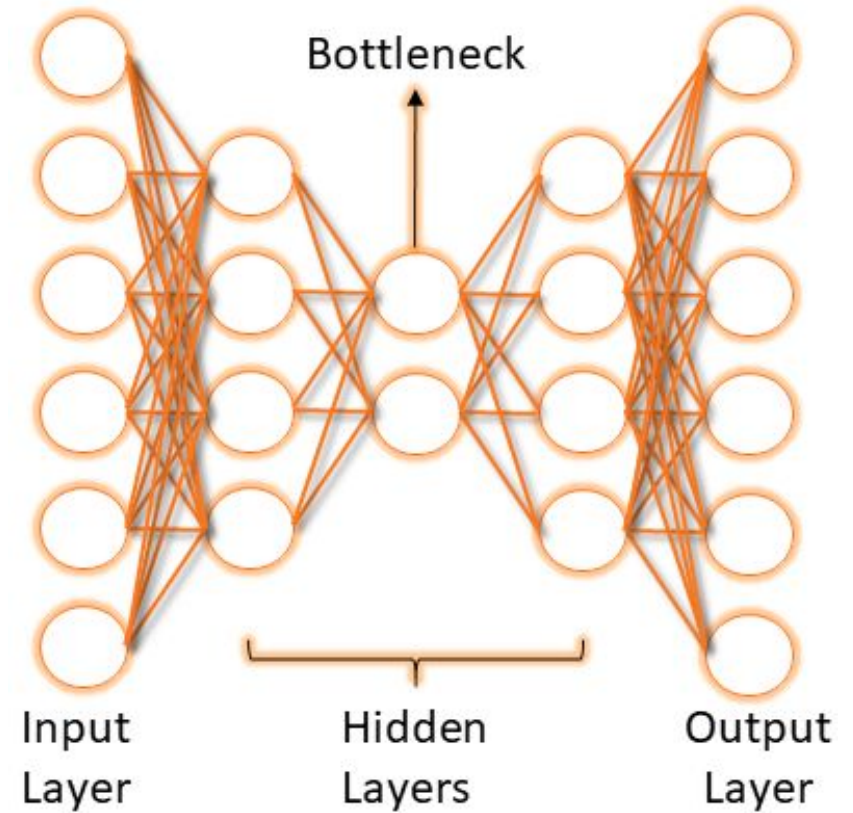
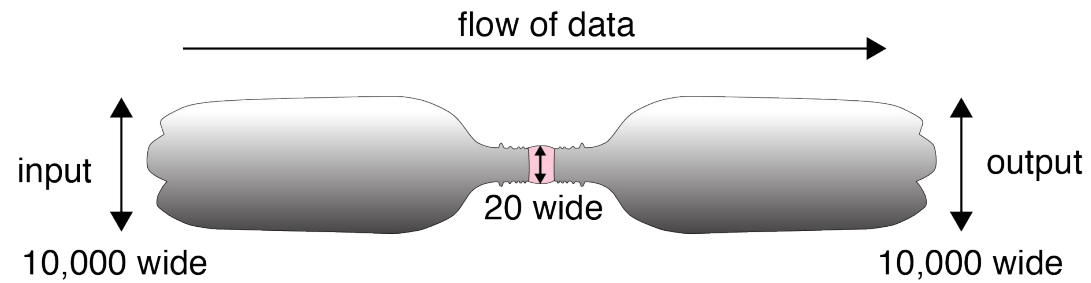
Decoder

- The decoder, on the other hand, takes the **encoded representation** produced by the **encoder** and **generates an output**.
- Generated output can be image or text or other.



Bottleneck

- We start with 10,000 elements
- We have 20 in the middle
- And 10,000 elements again at the end



Encoder – Decoder Architecture Application.

- Encoder – Decoder Architecture □ Examples.

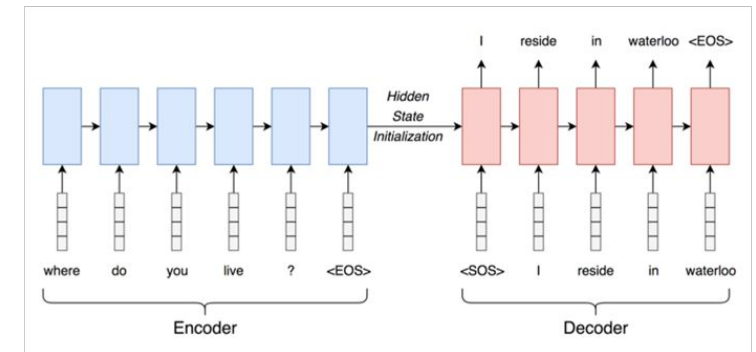
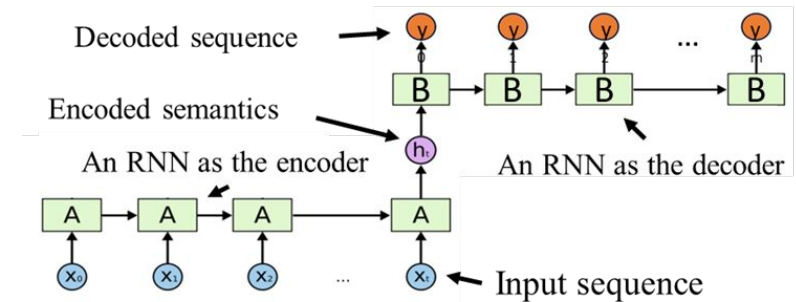
- Sequence – to Sequence Model {Neural Machine Translation}

- Encoder:

- The encoder turns each item into a corresponding hidden vector containing the item and its context.
 - extract** and **compress** the **semantics** from the **input sequence**
- An RNN in some variants like {LSTM or GRU is Used}

- Decoder:

- The decoder reverses the process, turning the vector into an output item, using the previous output as the input context.
 - generate** a **sequence** based on the **input semantics**
- An RNN in some variants like {LSTM or GRU is Used}
- { Week – 9's Discussion }



Encoder – Decoder Architecture Application.

- Encoder – Decoder Architecture □ Examples.

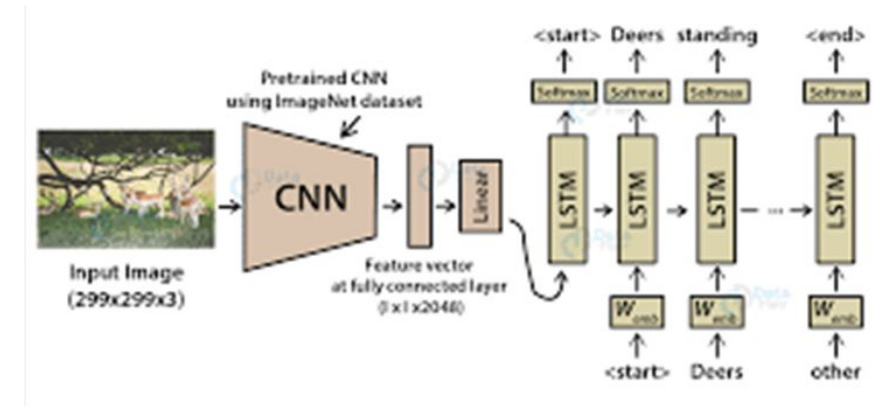
- Image Captioning

- Encoder:

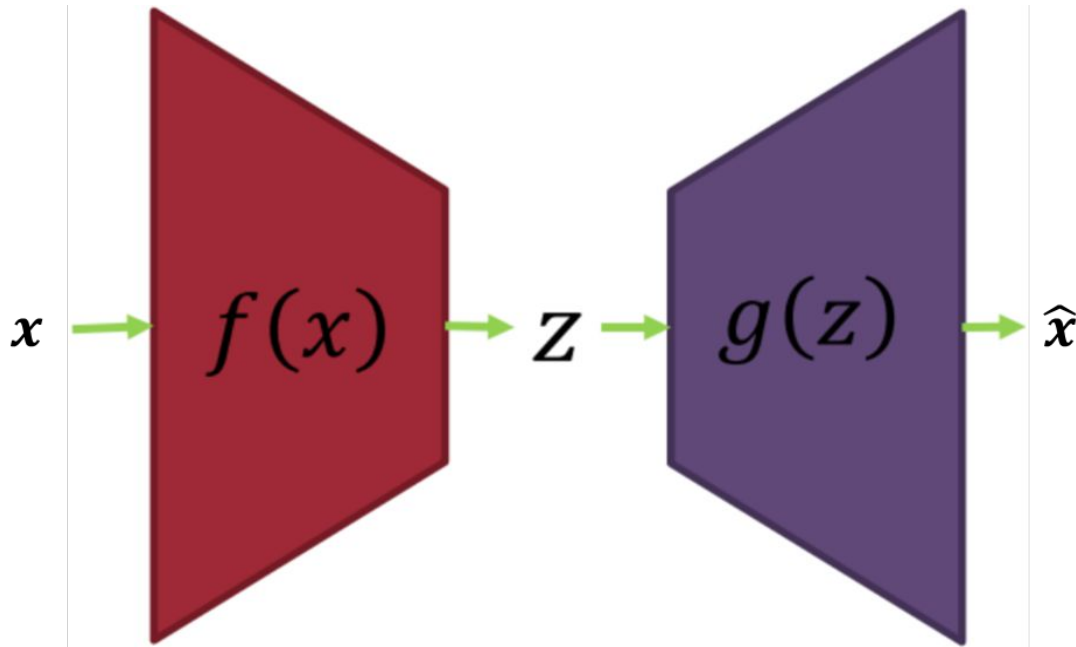
- The encoder turns each image into a corresponding hidden vector containing the discriminative features.
 - extract** and **compress** the **features** from the **input images**
- An **CNN** in some variants and with transfer learning is used.

- Decoder:

- The decoder reverses the process, turning the vector into an output item, using the previous output as the input context.
 - generate** a **sequence** based on the **input features**.
- An RNN in some variants like {LSTM or GRU is Used}



Idea-Autoencoder.



**Learning the identity function seems trivial!!!
Why and where can we use it?**

Given data X (no labels), we aim to learn the functions (encoder) and (decoder) where

$$f(x) = \text{act}(wx + b) = z$$

Here

act : activation functions.

z : latent representation or code:

$$g(z) = \text{act}(w'z + b') = h_{w,b}(x) = \hat{x}$$

$\hat{x}$ is x 's reconstruction.

$h(x) = g(f(x))$ approximation of identity function.

Vanilla Autoencoder.

- Fully-connected Autoencoder.
- Reconstruct high-dimensional data using a multilayer neural network with a narrow bottleneck layer.
- The bottleneck layer captures the compressed latent coding
- The low-dimensional representation can be used as the representation of the data in various applications.

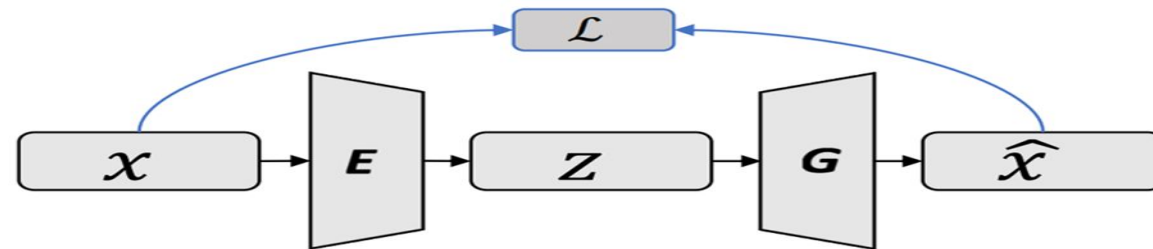
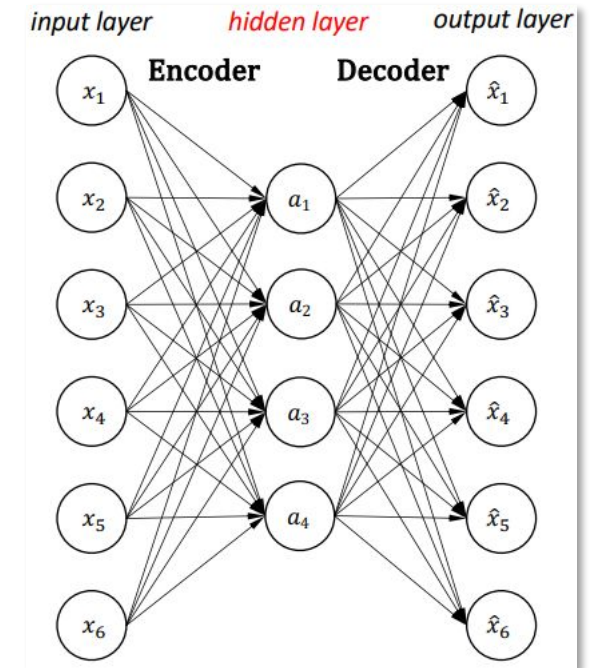


Fig: Vanilla Autoencoder.

Vanilla Autoencoder-Training.

- Ideally the input and reconstruction are identical.
- The loss can be distance between two, which can be measured by Mean Square Error:
 - $L(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^n (x^i - G(E(x^i)))^2$
- Where n : *number of variables*.
- Loss function is minimized using end to end backpropagation with stochastic gradient descent.
- If we do not use non-linear activation functions and minimize the MSE this is very similar to PCA.



If we can achieve the same with PCA, which is a kind of matrix factorization, more efficient than backpropagation + SGD. Why bother with autoencoders?

PCA Vs. Autoencoder.

PCA

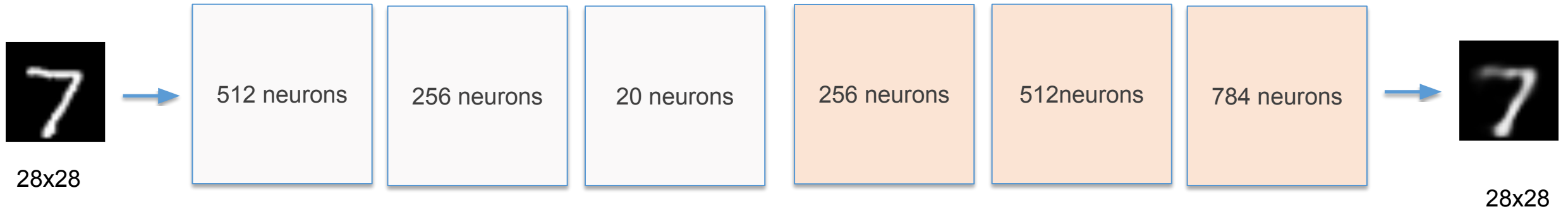
- PCA uses Eigen decomposition or Singular Value Decomposition (SVD) of the **covariance matrix** of the input data to perform dimensionality reduction.
- PCA is a linear dimensionality reduction technique. It can only learn linear relationships in the data

Autoencoders

- Autoencoder is a neural network-based architecture that is more complex than PCA.
- Autoencoder is a non-linear dimensionality reduction technique that can also learn complex non-linear relationships in the data.

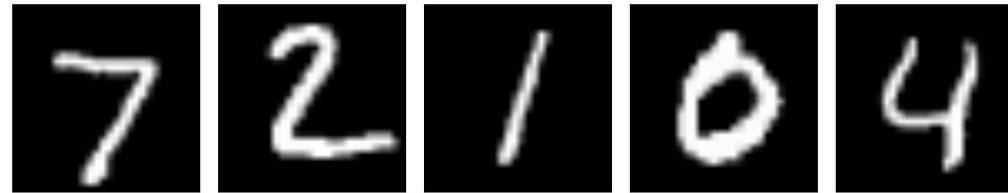
Deeper AutoEncoders

For a better representation we can add neurons in one layer or go deeper.

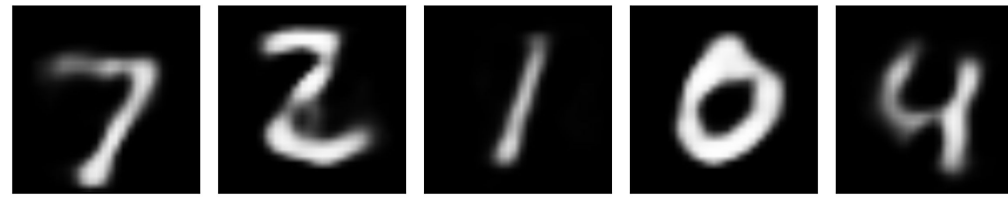


Z is 20 dimensional

Original Images



Reconstructed Images



Deeper 20 latent variables

Shallow 20 latent variables

DEEPER IS BETTER

Exploring autoencoders

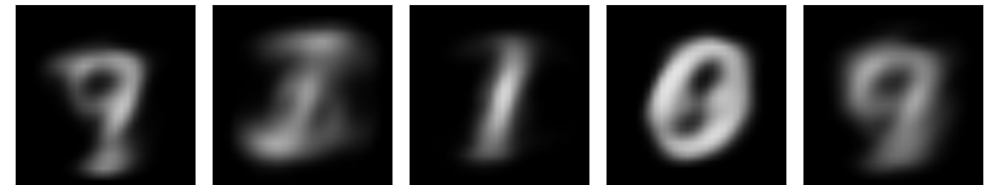
How can be sure that there is information in the latent space?

2 latent variables

original



reconstructed



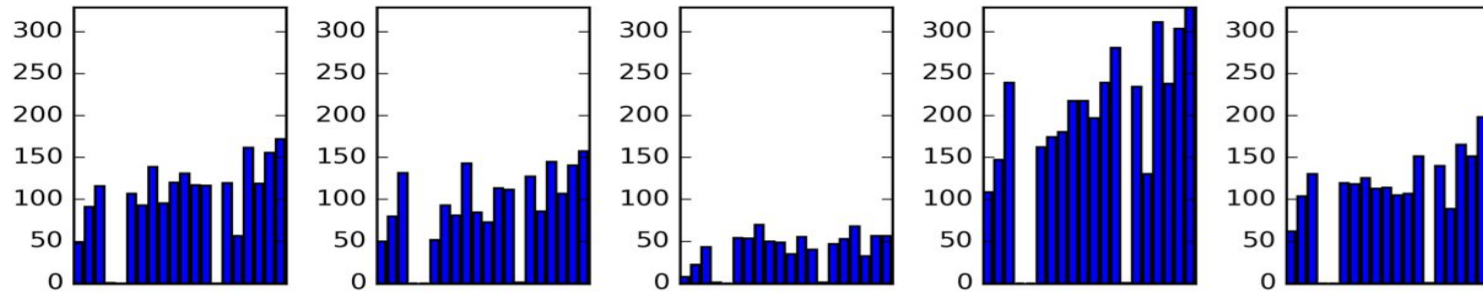
Exploring autoencoders (cont)

- For that let's explore the actual latent space.

Input



Latent space



Each plot a hist of the values of the 20 Z's

Output



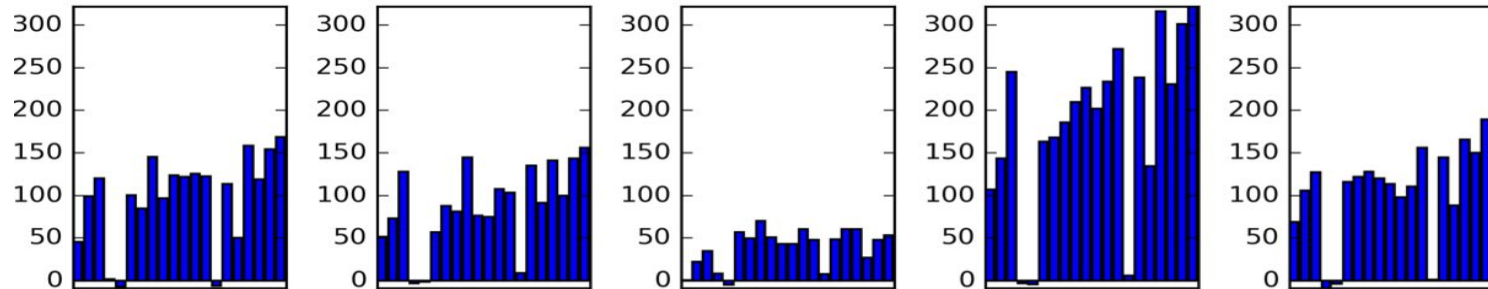
Exploring autoencoders (cont)

- If latent space does not contain any information, then changing the values randomly won't change the output or it will not be robust.

Input

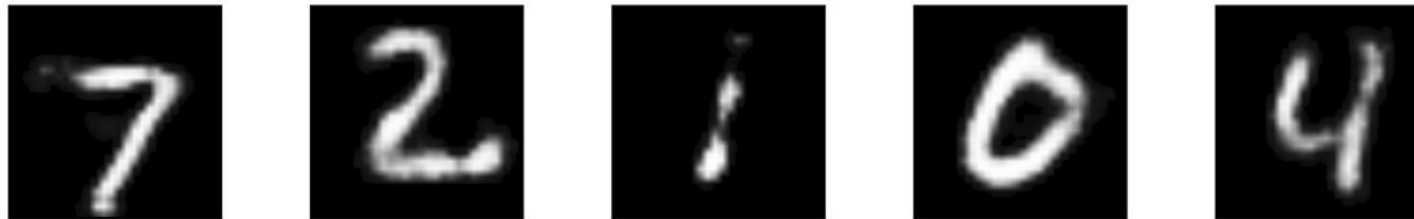


Latent space



Add random noise ± 10 to z 's

Output



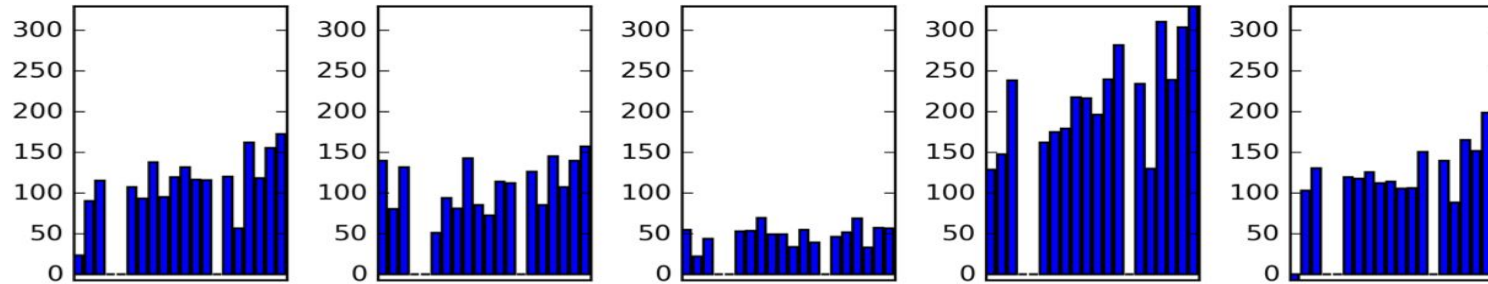
Exploring autoencoders (cont)

- If latent space does not contain any information, then changing the values randomly won't change the output or it will not be robust.

Input



Latent space



Add random noise ± 100 to z 's

Output

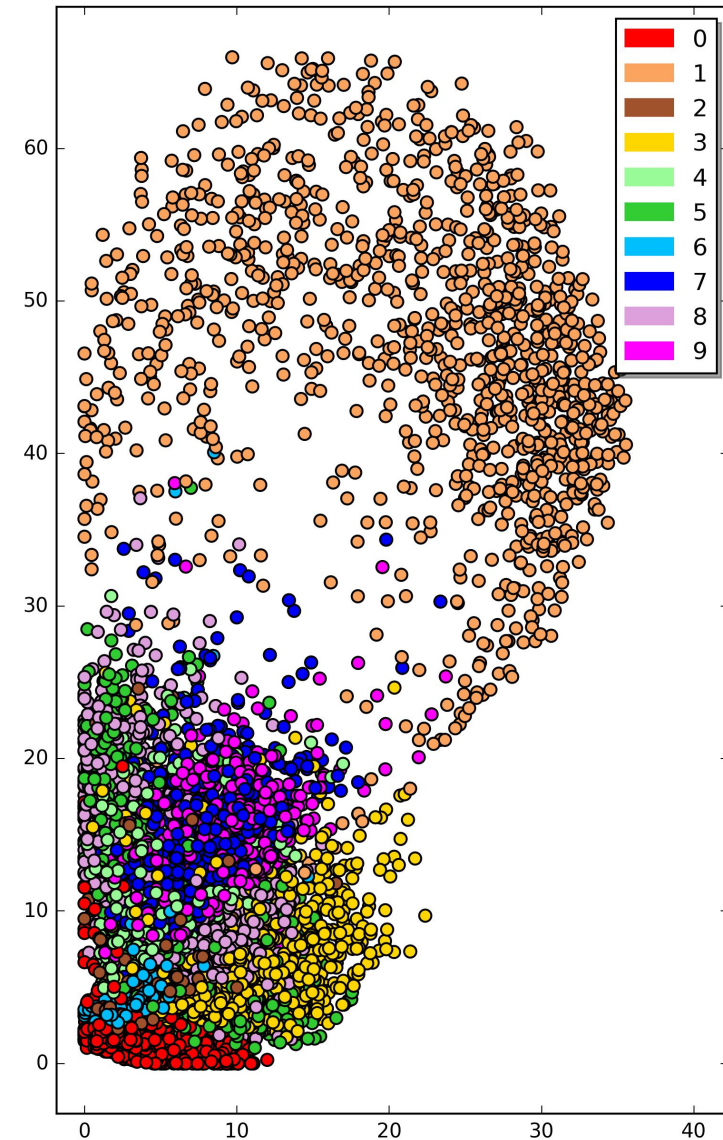


Parameter space of autoencoder

Further examine the latent space. Is there any separation of the different classes? If the AE learned the “essence” of the MNIST images, similar images should be close to each other.

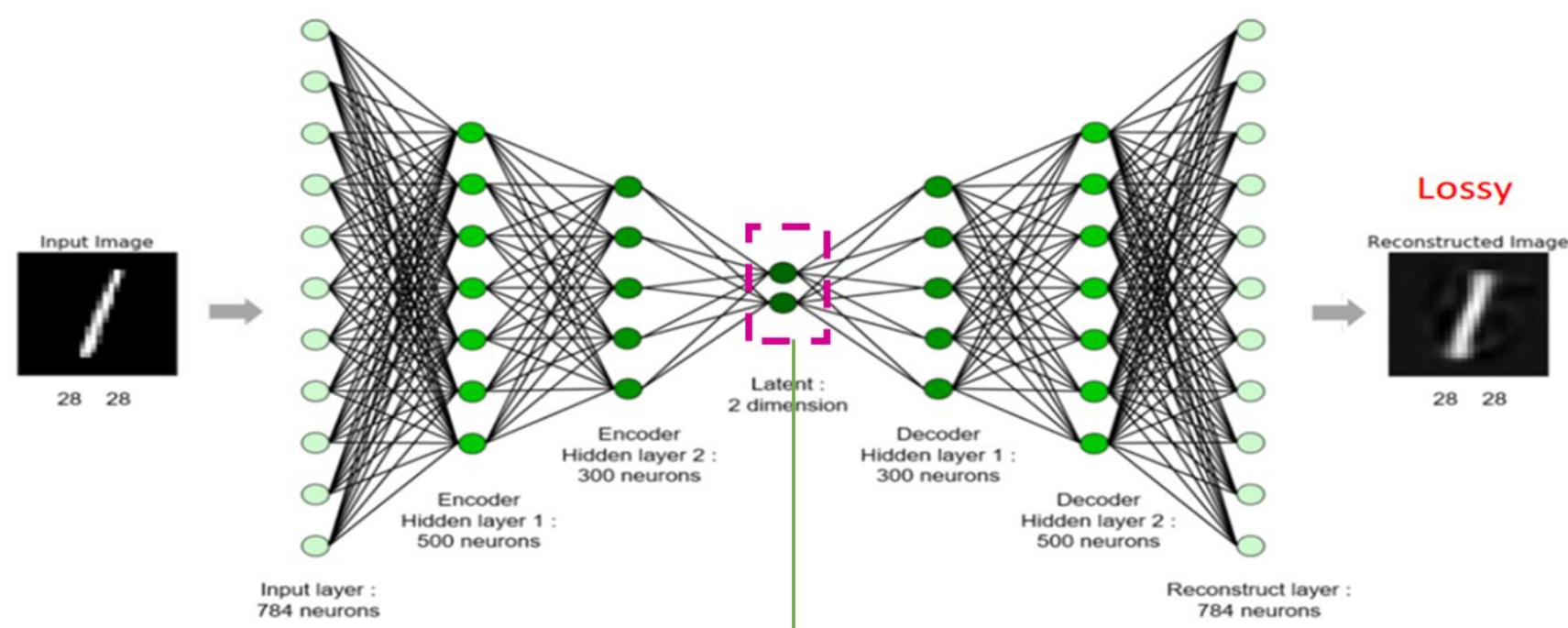
Plot the latent space and examine the separation.

Here we plot the 2 PCA components of the latent space.



~~Potential~~ Applications of Vanilla Autoencoder.

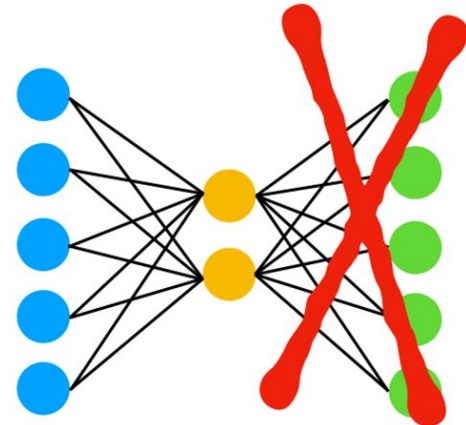
Vanilla Autoencoder: Dimensionality Reduction.



- Using an autoencoder with a **2-dimensional latent space**,
 - we reduce **the original 784-dimensional image representation** to just **2 dimensions**.
 - Each **image** is now encoded as a 2D point, capturing its most salient features.

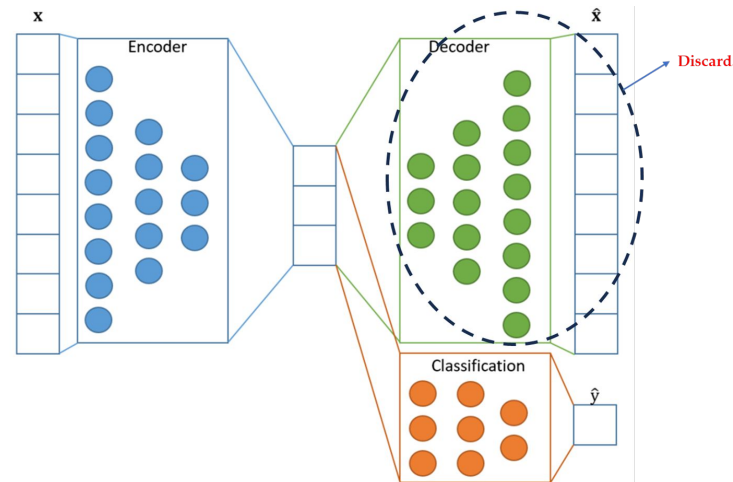
Vanilla Autoencoder: Feature Extraction.

- How it works as a Feature Extractor?
 - **Training the autoencoder:**
 - The model is trained to minimize the difference between the input and its reconstruction.
 - During this process, the **encoder** learns to capture the most informative features about the data in its latent space (i.e., the most important factors that explain the variability in the data).
 - **Using the Latent Representations:**
 - Once the model is trained, the encoder part is used to **extract features** from new data.
 - The latent space (encoded representation) becomes a **compressed version** of the input data,
 - containing the **relevant features for further tasks**.



Vanilla Autoencoder: Feature Extraction.

- **Feature Extraction for Downstream Tasks:**
 - **Classification:**
 - You can use the encoded features as inputs to another classifier (e.g., SVM, logistic regression).
 - **Clustering:**
 - The features can be clustered (e.g., using k-means) to identify groups within the data.
 - **Anomaly Detection:**
 - The reconstructed error can be used for anomaly detection, where a high error indicates that the input is an outlier.



Denoising Autoencoder: Working.

- Encode the Noisy Input:
 - $\mathbf{z} = \text{Encoder}(\tilde{\mathbf{x}}) \{ \tilde{\mathbf{x}} \rightarrow \text{Noisy Input} \}$
- Decode to Reconstruct the Clean Input:
 - $\hat{\mathbf{x}} = \text{Decoder}(\mathbf{z})$
- Train to minimize Reconstruction Loss:
 - $\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$
- Use Cases:
 - Image Denoising.
 - Robust Feature Extraction.

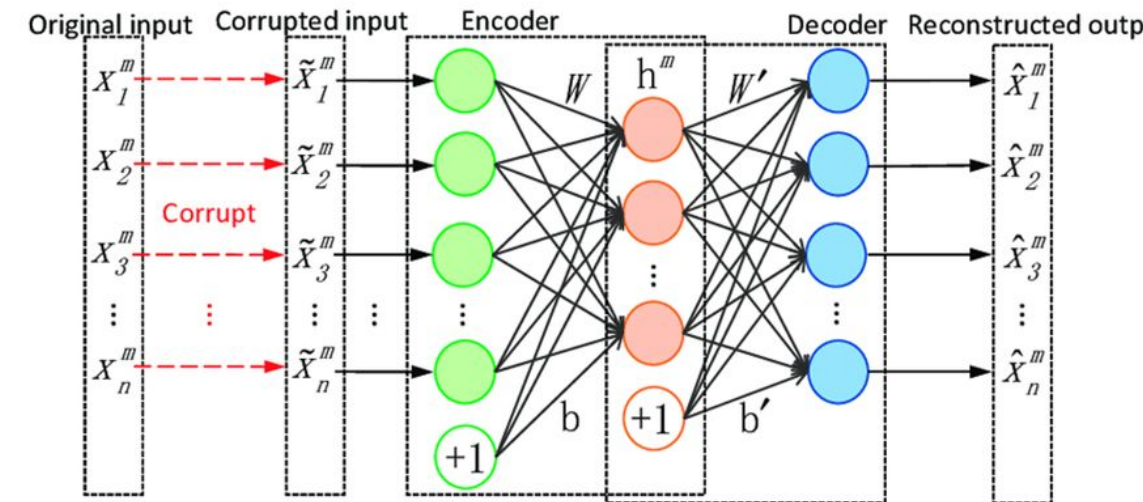


Image from Internet Subject to Copyright.

Denoising Autoencoder: Application.

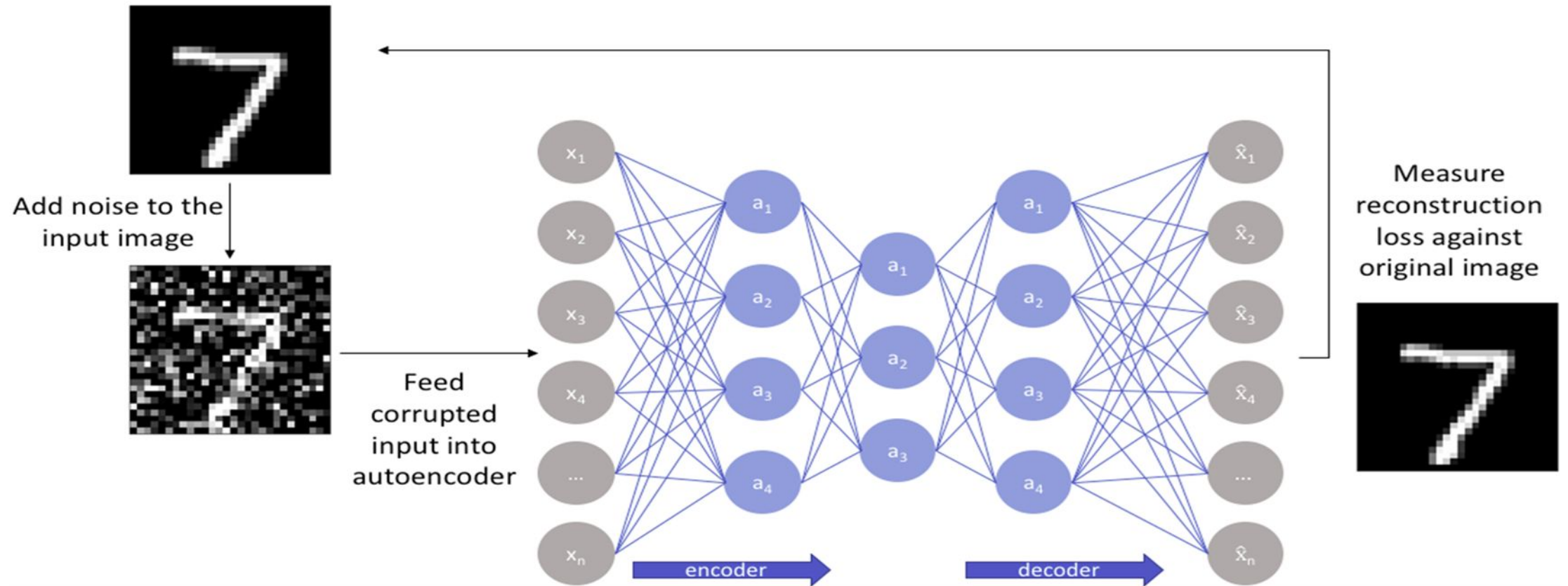


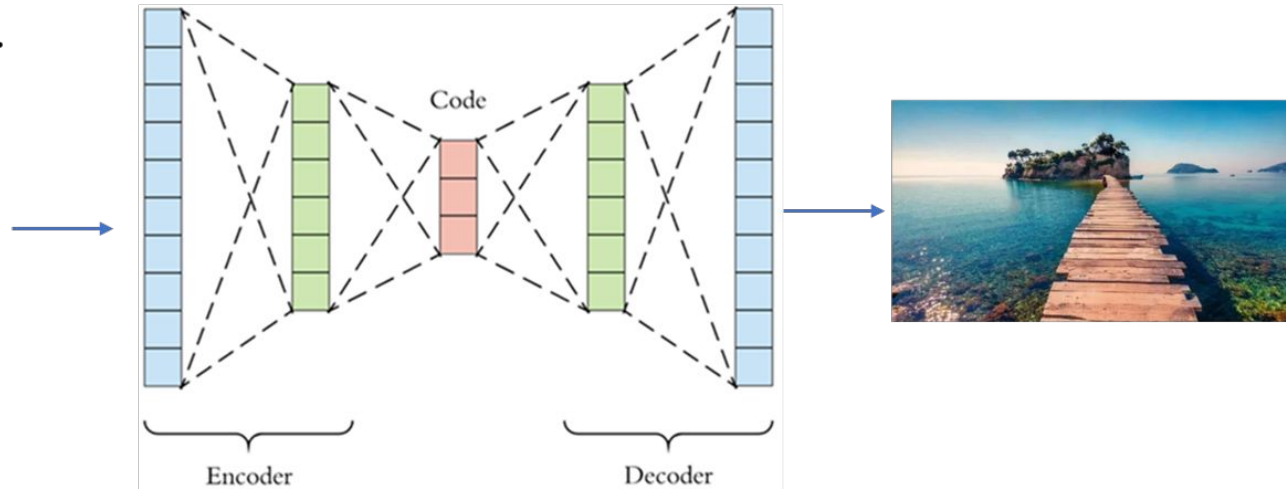
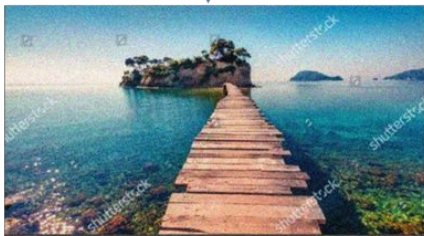
Image from Internet: Subject to copyright.

A Real-world application

- Water Mark Removal:

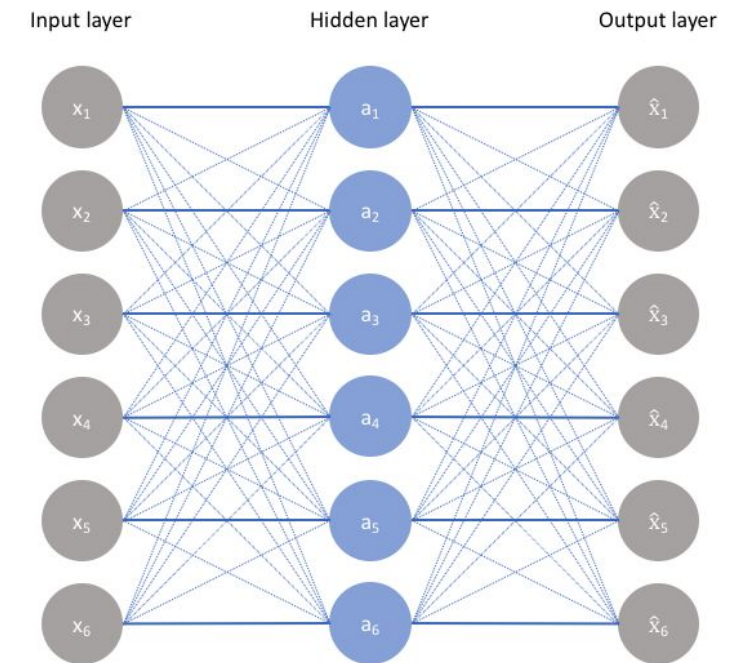


Add WaterMark and Noise.



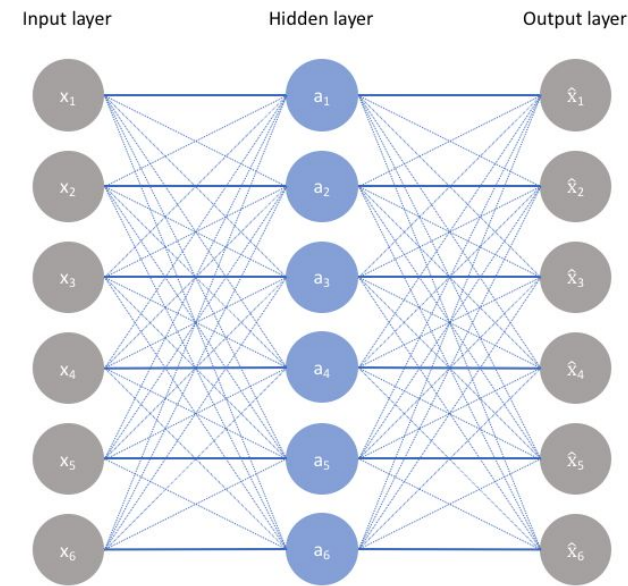
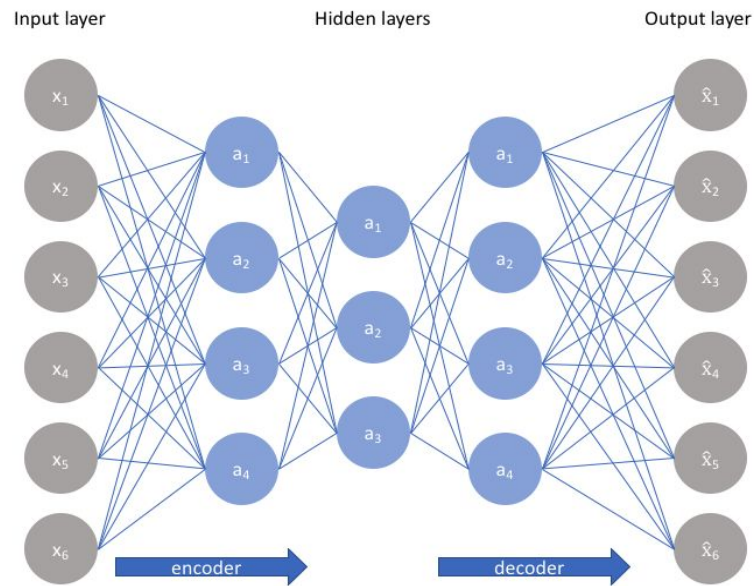
Overcomplete Autoencoders

- *Over-complete* autoencoder is when z has **greater dimension** than x .
- Autoencoder may simply copy input to output without learning anything useful
- The ideal autoencoder model balances the following:
 1. Sensitive to the inputs enough to accurately build a reconstruction.
 2. Insensitive enough to the inputs that the model doesn't simply memorize or overfits the training data.



Overcomplete Autoencoders

- We've assumed so far that the size of the bottleneck is smaller than the size of the inputs – this is called an **undercomplete autoencoder**.
- The case in which the size of the bottleneck is greater than or equal to the number of inputs we call an **overcomplete autoencoder**.



Sparse Autoencoders

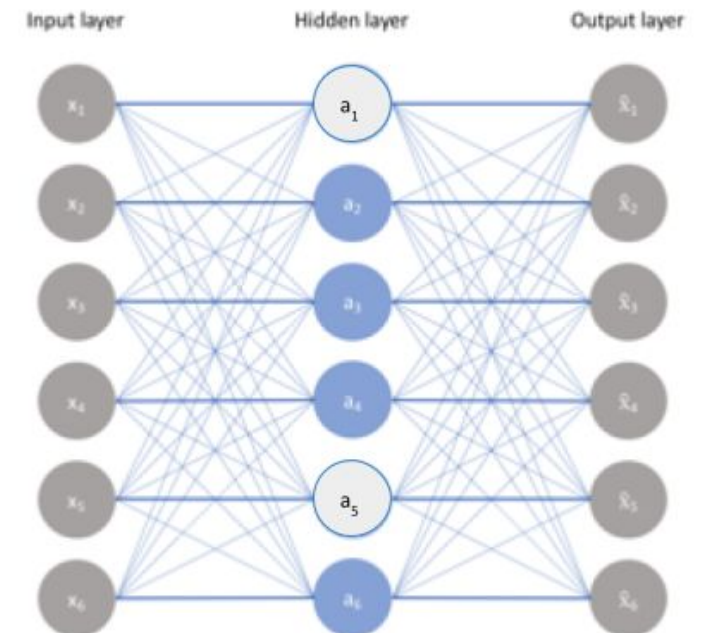
- This trade-off requires the model to maintain only the variations in the data required to reconstruct the input without holding on to redundancies within the input.
- **Question:** How to achieve this?
- For most cases, this involves constructing a loss function where one term encourages our model to be sensitive to the inputs (ie. reconstruction loss $\mathcal{L}(x, \hat{x})$ and a second term discourages memorization/overfitting (ie. an added regularizer).

$$\mathcal{L}(x, g(f(x))) + \Omega(z)$$

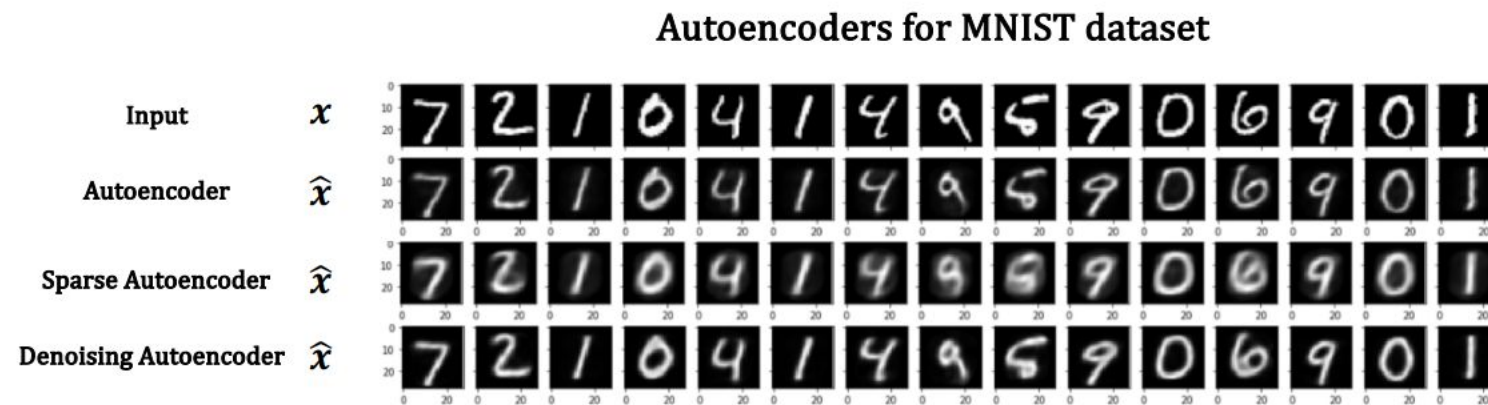
Sparse Autoencoders

- A sparse autoencoder is selectively activate regions of the network depending on the input data.
- Limiting the network's capacity to memorize the input data without limiting the networks capability to extract features from the data.

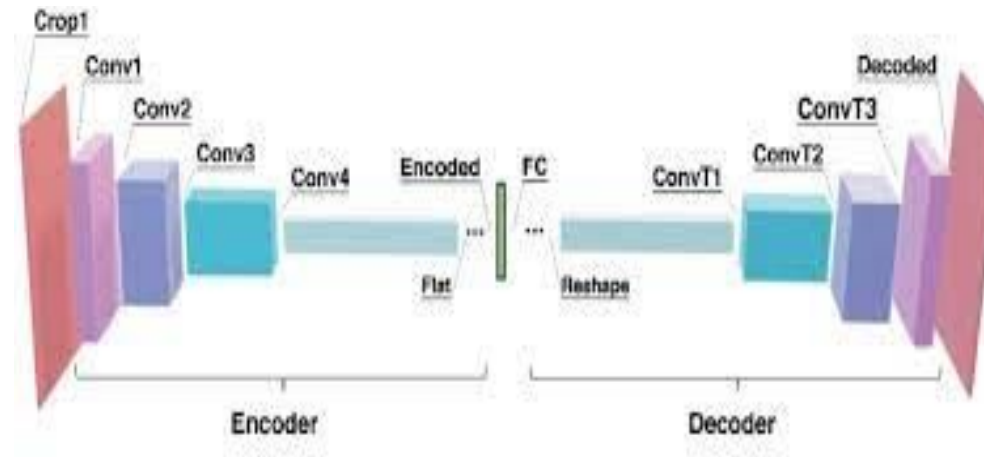
$$\mathcal{L}(x, \hat{x}) + \lambda \sum_i |z_i|$$



Vanilla Vs. Denoising Vs. Sparse.



Convolutional Autoencoder.



Convolutional Autoencoder.

- Encoder-
 - 1 or more convolutional layers.
- Decoder-
 - 1 or more Transposed convolutional layers.

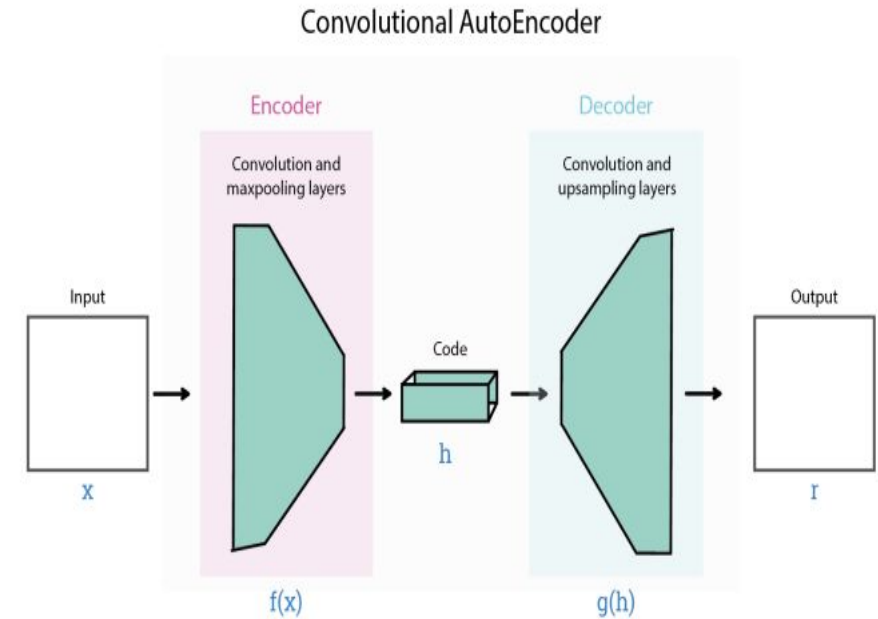


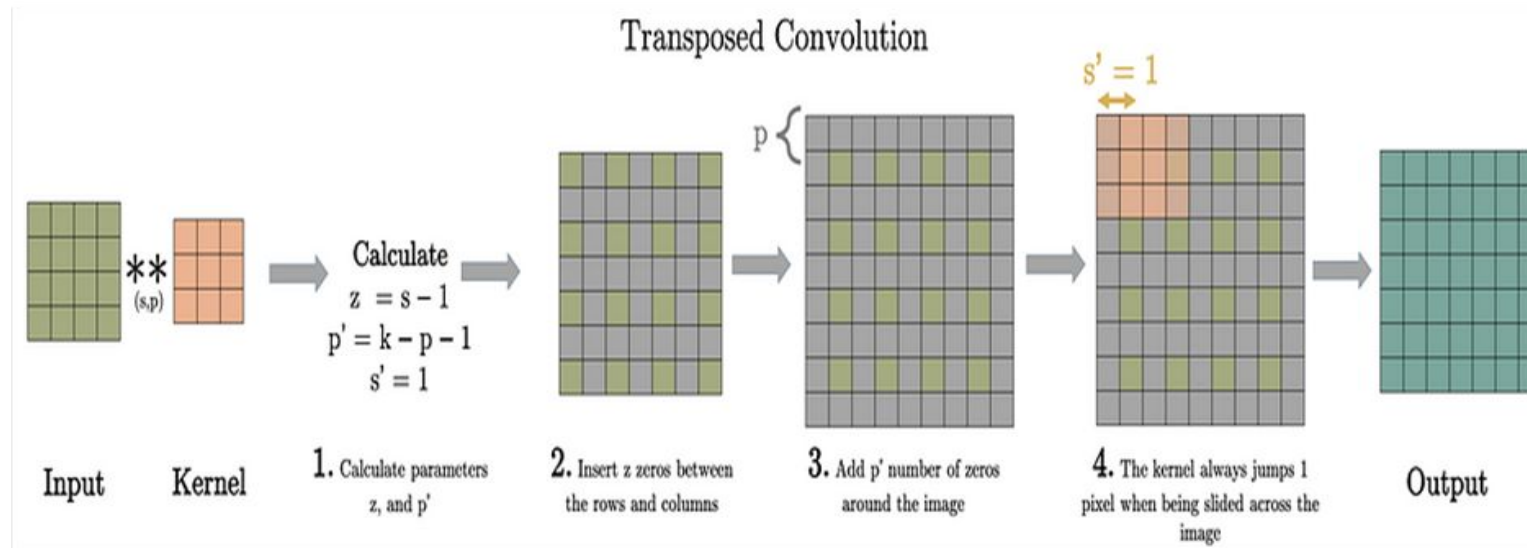
Fig. 3. General architecture of a convolutional autoencoder (CAE).

Transposed Convolutions.

- Allows us to increase the size of the output feature map compared to the input feature map.
- The output size of a transposed convolutional layer can be computed using the following formula:
 - $\text{output_size} = (\text{input_size} - 1) * \text{stride} + \text{kernel_size} - 2 * \text{padding}$
 - where:
 - input_size: the size of the input tensor along the spatial dimensions (width and height)
 - stride: the stride of the transposed convolution operation
 - kernel_size: the size of the transposed convolution kernel along the spatial dimensions
 - padding: the amount of zero padding added to the input tensor along the spatial dimensions
- Synonyms:
 - often also (incorrectly) called "**deconvolution**" (mathematically, deconvolution is defined as the inverse of convolution, which is different from transposed convolutions)
 - the term "unconv" is sometimes also used

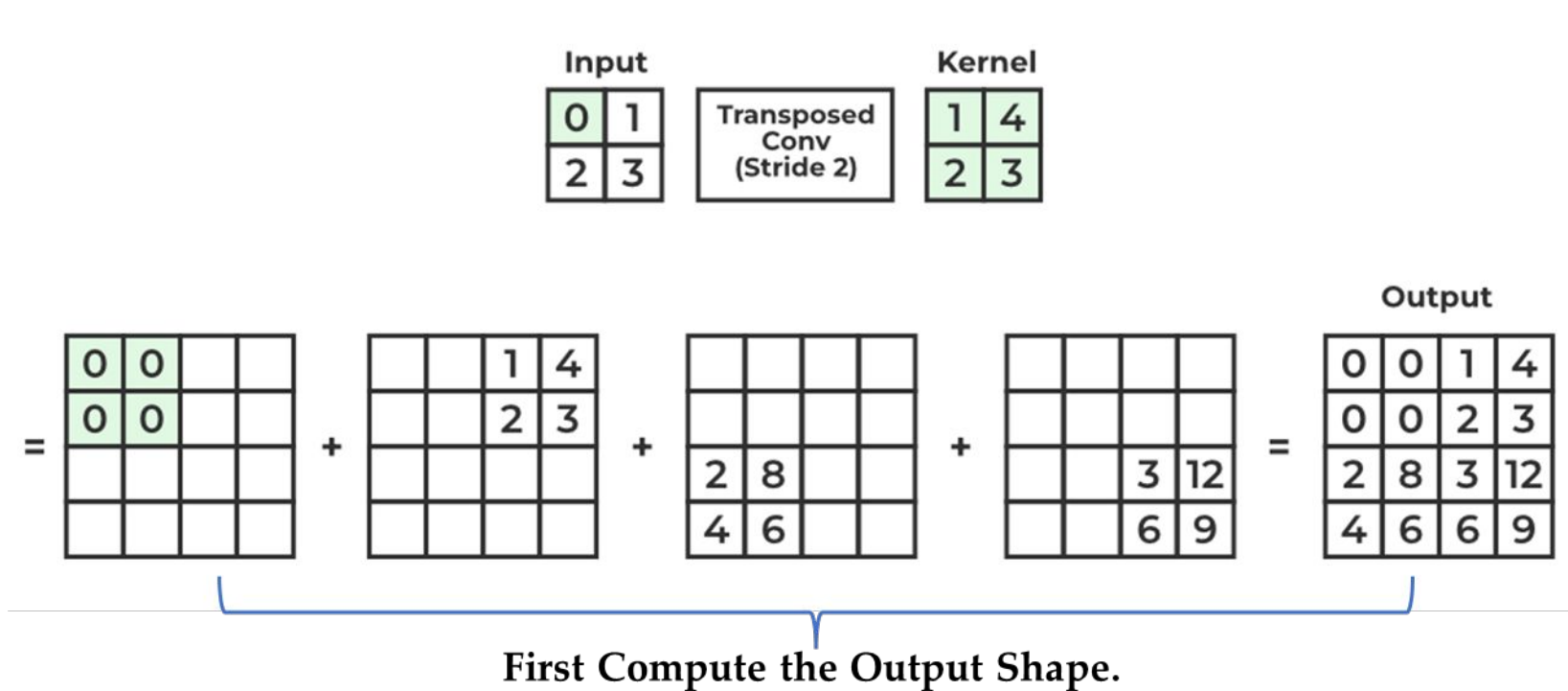
Transposed Convolutions – In Practice.

- Implementing a transposed convolutional layer can be better explained as a 4-step process:
 - Step 1:** Calculate new {hyper} parameters z and p'
 - Step 2:** Between each row and columns of the input, insert z number of zeros. This increases the size of the input to $(2*i-1) \times (2*i-1)$
 - Step 3:** Pad the modified input image with p' number of zeros
 - Step 4:** Carry out **standard convolution** on the image generated from step 3 with a **stride length of 1**



Transposed Convolutions-Demo.

- How Transposed Convolution Works?



Some Codes.

Vanilla Autoencoder.

```
# This is the dimension of the original space
input_dim = 10

# This is the dimension of the latent space (encoding space)
latent_dim = 2

encoder = Sequential([
    Dense(128, activation='relu', input_shape=(input_dim,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(latent_dim, activation='relu')
])

decoder = Sequential([
    Dense(64, activation='relu', input_shape=(latent_dim,)),
    Dense(128, activation='relu'),
    Dense(256, activation='relu'),
    Dense(input_dim, activation=None)
])
```

```
autoencoder = Model(inputs=encoder.input, outputs=decoder(encoder.output))
autoencoder.compile(loss='mse', optimizer='adam')
```

CAE.

```
input = layers.Input(shape=(28, 28, 1))

# Encoder
x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(input)
x = layers.MaxPooling2D((2, 2), padding="same")(x)
x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(x)
x = layers.MaxPooling2D((2, 2), padding="same")(x)

# Decoder
x = layers.Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same")(x)
x = layers.Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same")(x)
x = layers.Conv2D(1, (3, 3), activation="sigmoid", padding="same")(x)

# Autoencoder
autoencoder = Model(input, x)
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")
autoencoder.summary()
```

Model: "model"

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 28, 28, 1)]	0

conv2d (Conv2D)	(None, 28, 28, 32)	320

max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0

conv2d_1 (Conv2D)	(None, 14, 14, 32)	9248

max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 32)	0

conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 32)	9248

conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	9248

conv2d_2 (Conv2D)	(None, 28, 28, 1)	289
=====		
Total params: 28,353		
Trainable params: 28,353		
Non-trainable params: 0		

At the end!!!!

- Autoencoders learn data representation in an unsupervised/self-supervised way.
- Learned features are able to capture salient properties of data
- Different with vanilla autoencoder, in sparse autoencoder, the number of hidden units can be greater than the number of input variables.
- Can also be stacked to create “deep” autoencoders.

Implementation will be done in the tutorial, please bring your laptop!

Thank You!!!!